

Computer Hardware

Introduction to Processor Architecture

Ksawery Moźdzynski

Agenda For Today



1. What Is a Computer?

Input, programs, processing, output, and programmability



2. Designing a Computer

Five design decisions behind the von Neumann model



3. How Does a CPU Work?

Processor functions, registers, ALU, CU, and the instruction cycle

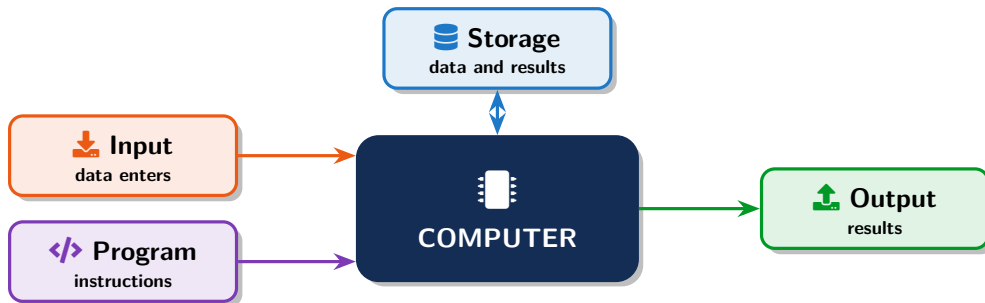


4. Programming a Simple CPU

MARIE.js, assembly instructions, input/output, jumps, and loops

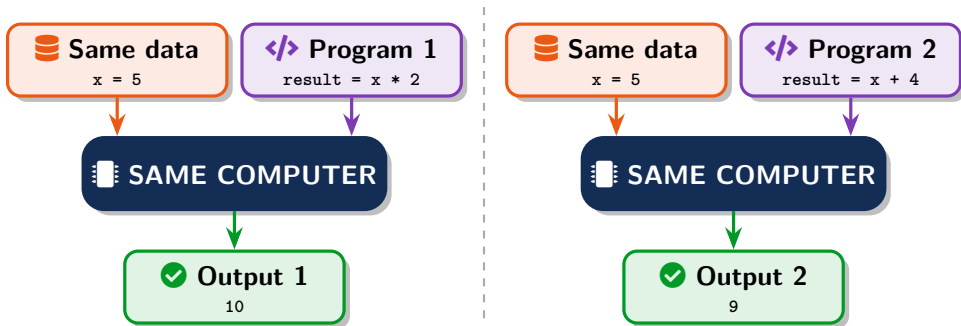
Formal definition

A computer is a **programmable** device that accepts **data as input**, processes it according to **a set of instructions (a program)**, and **produces output**. It usually also **stores data and results**.



The central idea: programmability

Programmability means that the same hardware can perform different tasks: given the **same data**, different **programs** make the computer produce different **results**.



Is this definition precise enough?

“A programmable device that takes input, processes it according to a program, and produces output” tells us **what a computer does**, but not **how it is organised**.

	Design dilemma	Possible answers
1	Where does the program live?	Wired program; Program on tape; Program in memory
2	Do instructions and data live in the same place?	Two separate memories; One shared memory
3	How do we represent information?	Decimal numbers, special codes; Everything as binary numbers
4	What runs next?	Many at once; When the data is ready; One after another
5	How is the machine organised?	One undivided machine; Separate units

We need a more precise model

Different answers give different computers.

Let's design our own computer model

A computer architect's perspective

Before looking at the final model, let us step into the role of a computer architect and make the key design decisions.

Our approach

We will analyse the design choices embodied in the von Neumann model.

Dilemma 1: where does the program live?

Design question: how can one machine run many different programs without being rebuilt each time?

Wired program

🔧 fixed circuit
program = wiring

- + very fast and highly optimised for one fixed task
- a new task means physical rewiring

Program on tape



- + swap the tape to change the task
- slow mechanical reading; loops and jumps are awkward

Program in memory

LOAD 6	0
ADD 7	1
STORE 8	2

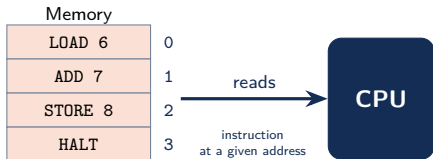
- + fast electronic access to any instruction
- needs enough memory and a way to load the program

Which would you choose, and why?

(1 Stored program) (2 Shared memory) (3 Binary) (4 Sequential) (5 Separate units)

Decision 1: keep the program in memory

Von Neumann's answer: the program is stored in the computer's **memory**, as numbered instructions.



What it gives us

- One machine, many programs: a new task means new contents of memory.
- Any instruction is reachable by its address, so jumps and loops are easy.

What it costs: enough memory, and a way to load the program into it.

The program is now stored. Next question: where are the data stored?

1 Stored program

2 Shared memory

3 Binary

4 Sequential

5 Separate units

Dilemma 2: where do instructions and data live?

Design question: should instructions and data use the same memory, or should they have separate memories?

Separate memories

program memory		data memory	
LOAD 0	0	5	0
ADD 1	1	3	1
HALT	2	0	2

address 0 exists twice

- + instructions and data can be accessed independently
- two memories, two address spaces, and more complex organisation

One shared memory

LOAD 6	0
ADD 7	1
:	
5	6
3	7

one numbering

- + one address space and one simple memory model
- instructions and data compete for the same path

Which design would you choose, and why?

1 Stored program

2 Shared memory

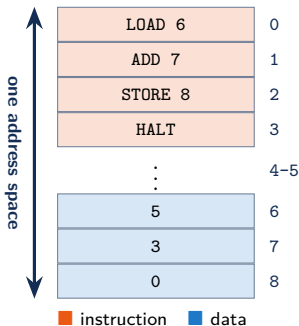
3 Binary

4 Sequential

5 Separate units

Decision 2: one memory for instructions and data

Von Neumann's answer: there is **one memory**. Instructions and data are stored side by side and share **one numbering of cells** (one address space).



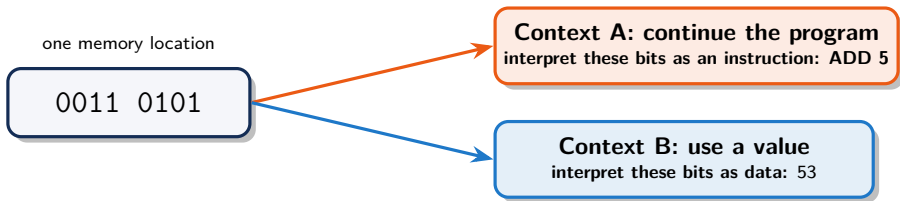
What does “the same” mean? Three things:

- **Same storage.** One memory, not a program memory plus a separate data memory.
- **Same addresses.** One numbering 0, 1, 2, ...; each address belongs to exactly one cell.
- **Same kind of cell.** A cell is a group of bits. There is no special kind of cell for instructions.

The price: instructions and data travel over the same path (the von Neumann bottleneck, later).

Decision 2 (continued): how do bits get their meaning?

Looking at memory alone, we see addresses and bit patterns, not labels such as “instruction” or “data”. **The program context gives the bits their meaning.**



Key idea: memory stores bit patterns. It does not store their meaning as a label.

Important: the same kind of memory cell can hold an instruction or a value, depending on the program context.

1 Stored program

2 Shared memory

3 Binary

4 Sequential

5 Separate units

Dilemma 3: how do we represent information?

Design question: in what form are numbers and instructions kept inside the machine?

Decimal numbers, special codes

digits 0–9

instructions: other code

- + close to how people write numbers (early machines such as ENIAC worked in decimal)
- more complex circuits (ten states per digit); numbers and instructions need different formats

Everything as binary numbers

0000 0101 (5)

0011 0111 (ADD 7)

- + two-state circuits are simple and reliable; instructions and data share one format
- people must convert; long strings of bits

Which would you choose, and why?

1 Stored program

2 Shared memory

3 Binary

4 Sequential

5 Separate units

Decision 3: everything is binary

Von Neumann's answer: both instructions and data are **binary numbers**: fixed-size groups of bits.

An instruction is a number too

opcode	address
0001	0110
LOAD	cell 6
Opcode	Operation
0001	LOAD
0010	STORE
0011	ADD
0111	HALT

Our example program, as it sits in memory

Address	Bits	Meaning
0	0001 0110	instruction LOAD 6
1	0011 0111	instruction ADD 7
2	0010 1000	instruction STORE 8
3	0111 0000	instruction HALT
6	0000 0101	number 5
7	0000 0011	number 3
8	0000 0000	number 0 (result)

8-bit teaching format: 4-bit opcode + 4-bit address (opcodes as in MARIE, used later). Real CPUs use longer, richer formats.

Gain: one kind of hardware stores, copies and moves both. **Price:** people need tools (assemblers, compilers) to write instructions comfortably.

1 Stored program

2 Shared memory

3 Binary

4 Sequential

5 Separate units

Dilemma 4: how is instruction execution controlled?

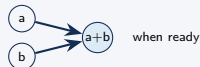
Design question: should the computer follow the program step by step, or can it execute independent operations at the same time?

Parallel execution



- + independent operations can run together
- harder to design and coordinate

Data-driven execution



- + an operation runs when its inputs are ready
- complex hardware and less predictable order

Sequential control



- + instructions follow the order of the program
- less parallelism at the program level

Which would you choose, and why?

1 Stored program

2 Shared memory

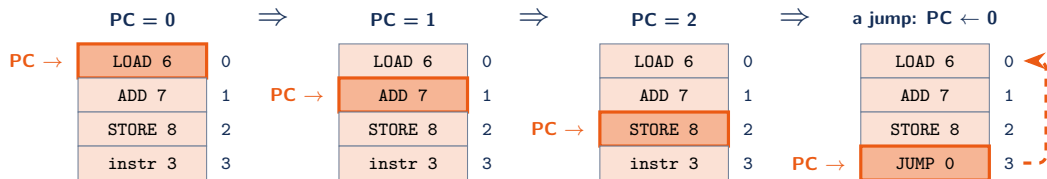
3 Binary

4 Sequential

5 Separate units

Decision 4: an ordered program sequence

The von Neumann model: the program is represented as an **ordered sequence of instructions**. The **program counter (PC)** remembers the address of the next instruction; a jump simply writes a new address into it.



Gain: a predictable program order. Jumps make loops and conditions possible.

Important: modern CPUs may overlap independent instructions internally, while preserving the program's visible order.

1 Stored program

2 Shared memory

3 Binary

4 Sequential

5 Separate units

Dilemma 5: how is the machine organised?

Design question: should the computer be one undivided whole, or a set of parts with separate jobs?

One undivided machine

one block:
everything mixed together

- + can be tuned for a single task
- hard to understand, change or reuse: every change touches everything

Separate units



one job each

- + each part has one clear job and can be designed and improved on its own
- the parts must be connected to each other

Which would you choose, and why?

1 Stored program

2 Shared memory

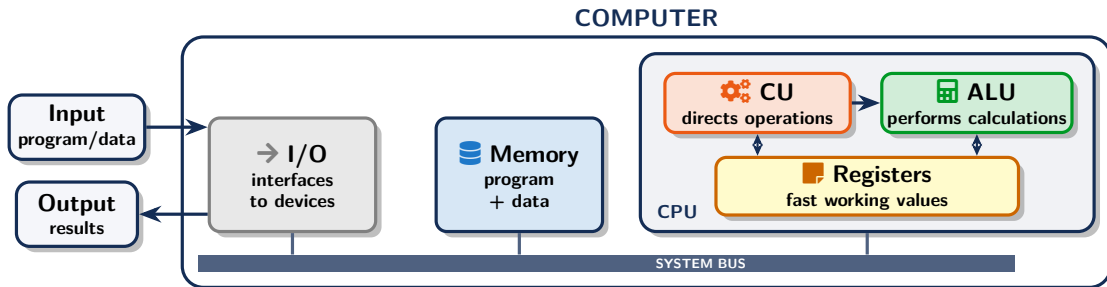
3 Binary

4 Sequential

5 Separate units

Decision 5: separate functional units

Von Neumann's answer: separate functional units for memory, control, computation, registers, and I/O.



Input and output enter through the I/O subsystem; program and data live in Memory; a bus connects the units.

Task: explore alternative design choices

Choose one design or extension and investigate why it is useful.



Memory organisation

Harvard architecture

Separate instruction and data memories.



Data processing

SIMD architecture

One operation applied to many data values. Used in graphics and image processing.



Execution control

Dataflow architecture

Operations run when their inputs are ready.

Compare → Explain → Evaluate

1. Compare

Which design choice differs from von Neumann?

2. Explain

What problem does this design solve, and where is it used?

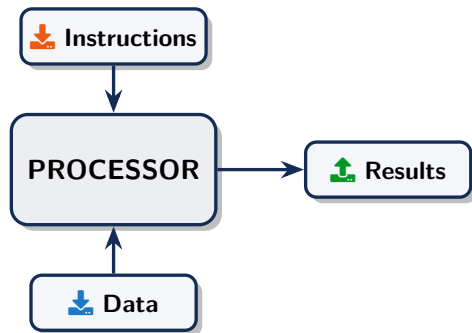
3. Evaluate

What is one advantage and one limitation?

What does a processor do?

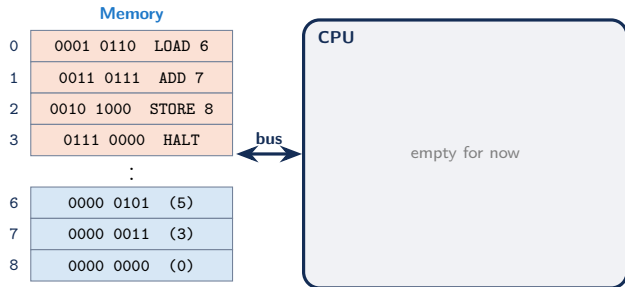
A processor is the active part of a computer: it follows instructions and transforms data into results.

- 1 **Read instructions** – find out what needs to be done.
- 2 **Understand instructions** – identify the operation.
- 3 **Get data** – obtain the values needed for it.
- 4 **Perform the operation** – calculate, compare, transform.
- 5 **Store the result** – keep it, or send it onward.



First understand the jobs. Next, we will meet the hardware parts that perform them.

What does the CPU work with?



The program and its data are in memory. The CPU is connected to memory by the bus and can do only two things with it:

Read

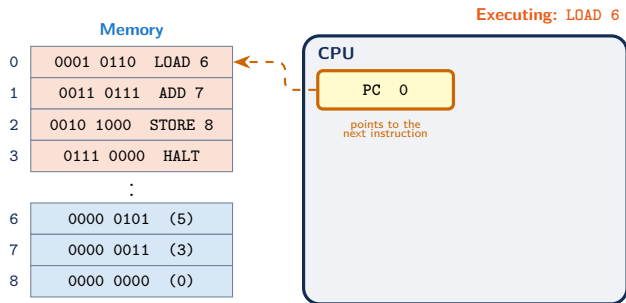
The CPU can **read one cell** from memory.

Write

The CPU can **write to one cell** in memory.

The CPU does **not** see the whole program.
At any moment it gets **only one cell**.

The program counter (PC)



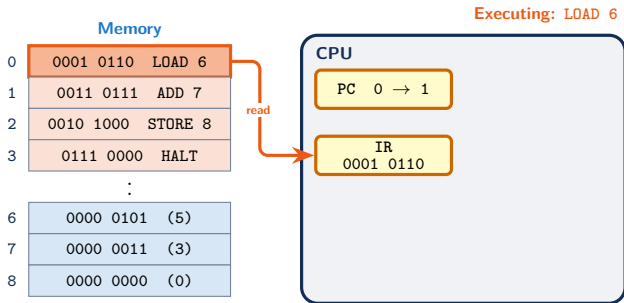
The **program counter (PC)** is a register that points to the **next instruction** the CPU will execute.

Register
a small, very fast storage cell *inside* the CPU.

Without the PC, the CPU would not know which cell to read.

At the start: PC = 0.

The instruction register (IR) and fetch



The CPU reads the cell the PC points to: **cell 0**.
Memory sends back 0001 0110.

The **instruction register (IR)** holds the instruction the CPU is executing now.

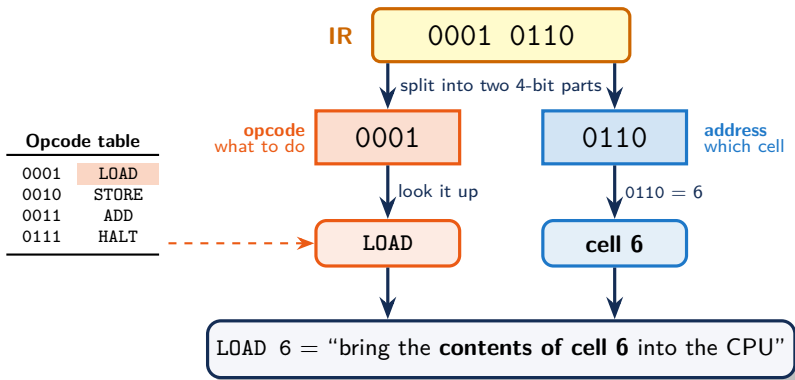
The CPU keeps a copy because it needs the instruction in the next steps (decode and execute).

Then the **PC increases by 1**: it already points to the next instruction.

Name of this phase: **fetch**

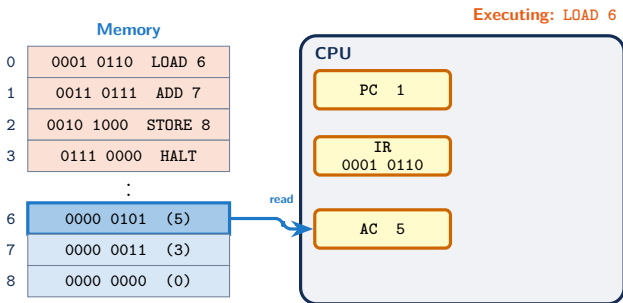
Decoding: what do these bits mean?

The IR holds 0001 0110. Our CPU splits every instruction into two 4-bit parts.



Name of this phase: **decode**

The accumulator (AC) and execute



The CPU reads **cell 6**. Memory sends back 0000 0101 = 5.

The instruction said **6**, the value we got is **5**: 6 was the number of the cell.

The **accumulator (AC)** is a register that holds the working value: the number the CPU is working with right now.

The CPU keeps the 5 in the AC because the next instruction (ADD 7) will need it.

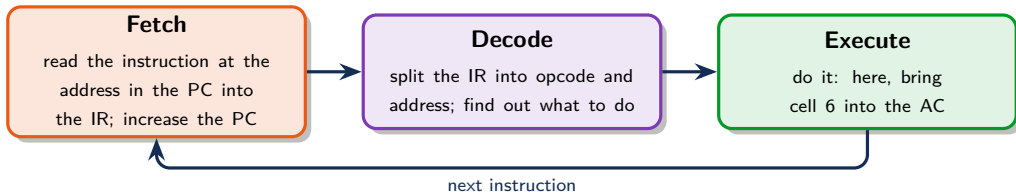
Nothing was calculated: LOAD only moves a value.

Name of this phase: **execute**

State: PC = 1, IR = LOAD 6, AC = 5

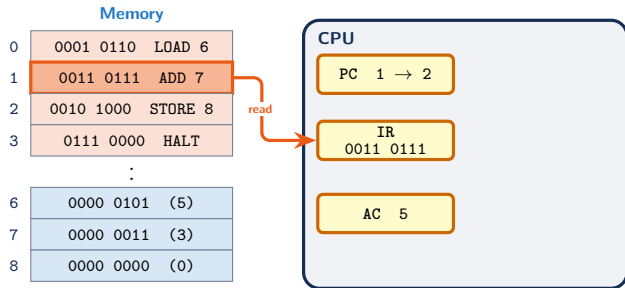
What did we just do?

One instruction took **three phases**. Now the CPU simply repeats them for the next instruction.



The PC already holds 1, so the CPU is ready for the next instruction: ADD 7.

ADD 7: fetch and decode



Fetch

PC = 1: read cell 1
IR ← 0011 0111
PC → 2

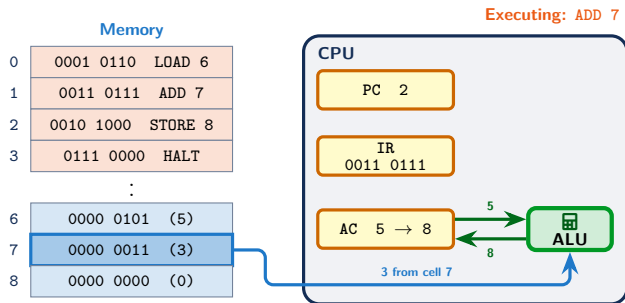
Decode

0011 = ADD
0111 = cell 7

The same steps as before, only different bits.

ADD 7 names only one cell. The other number
always comes from the AC, which still holds 5.

The arithmetic logic unit (ALU)



ADD 7 needs two numbers: **5** from the AC and **3** from cell 7.

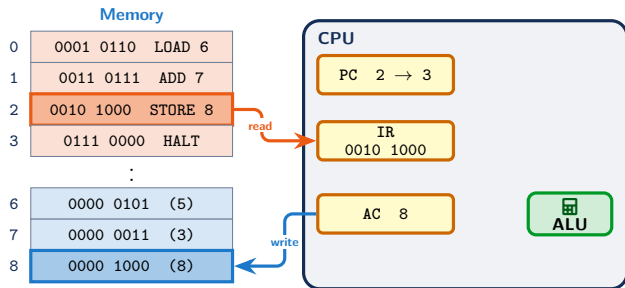
The CPU has a **specialised unit** for calculations: the **arithmetic logic unit (ALU)**.

Both numbers go into the ALU. The result goes back to the AC.

Name of this phase: **execute**

State: PC = 2, IR = ADD 7, AC = 8

STORE 8: data goes the other way



Fetch

PC = 2: read cell 2; IR $\leftarrow$ 0010 1000; PC $\rightarrow$ 3

Decode

0010 = STORE; 1000 = cell 8

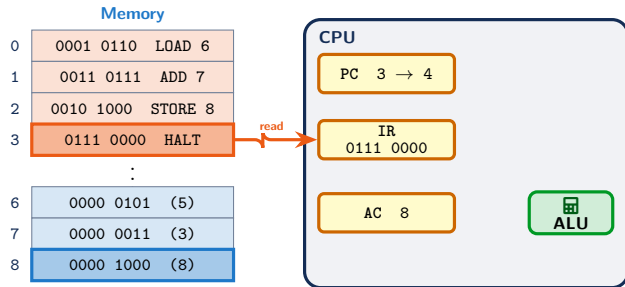
Execute

write the AC into cell 8: data goes **from the CPU to memory**.

Cell 8: 0000 0000 $\rightarrow$ 0000 1000

The first moment the program **changes** memory.

HALT: the end of the program



Fetch

PC = 3: read cell 3; IR $\leftarrow$ 0111 0000; PC $\rightarrow$ 4

Decode

0111 = HALT

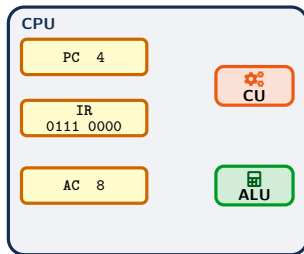
Execute

the CPU stops repeating the cycle.

Result: cell 8 holds $0000\ 1000 = 8 = 5 + 3$.

Who was directing all of this?

We kept saying “the CPU reads”, “the CPU decodes”, “the CPU increases the PC”. The part of the CPU that does these things in the right order is the **CU (control unit)**.



Problem	Solution	Box in Decision 5
Which cell to read next?	PC	Registers
Where to keep the instruction?	IR	Registers
What do the bits mean?	decoding	CU
Where to keep the working value?	AC	Registers
Who calculates?	ALU	ALU
Who runs the whole sequence?	CU	CU

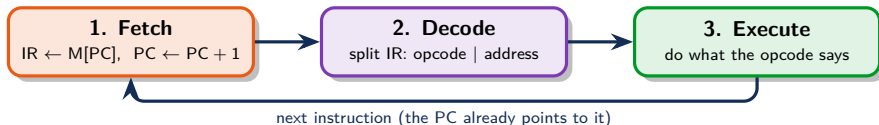
The **CU** decides and coordinates, but never calculates.

The **ALU** calculates, but never decides what to do.

The instruction cycle: fetch, decode, execute

What we have just seen four times, written compactly: the CPU repeats **the same three phases** for every instruction, until it meets HALT.

Notation: $M[a]$ means “the contents of memory cell number a ” (e.g. $M[6] = 5$); $\leftarrow$ means “is written into”.



What “execute” means for each opcode

Instruction	Effect	Needs
LOAD a	$AC \leftarrow M[a]$	read memory
ADD a	$AC \leftarrow AC + M[a]$	read memory + ALU
STORE a	$M[a] \leftarrow AC$	write memory
HALT	stop the cycle	nothing

Fetch and decode are identical for every instruction. Only **execute** differs.

The whole run in one table

One row per instruction: which cell was read, what it meant, and the state afterwards

PC (cell read)	Bits	Decoded instruction	AC after	M[8] after
0	0001 0110	LOAD 6	5	0
1	0011 0111	ADD 7	8	0
2	0010 1000	STORE 8	8	8
3	0111 0000	HALT	8	8

Result: when the CPU stops, $M[8] = 8 = 5 + 3$.

Task: you are the CPU

Individual task (12 min). The memory below holds a program and its data, exactly as the CPU sees it. Fetch, decode and execute it by hand.

Memory	
Addr	Bits
0	0001 0110
1	0011 0111
2	0011 0110
3	0010 1000
4	0111 0000
5	0000 0000
6	0000 0100
7	0000 1001
8	0000 0000

Opcode table	
Opcode	Instruction
0001	LOAD a
0010	STORE a
0011	ADD a
0111	HALT

first 4 bits = opcode,
last 4 bits = address a

At the start PC = 0:
the program starts at cell 0.

Fill in one row per instruction

PC (cell read)	Bits	Decoded instruction	AC after	M[8] after
0				

Then answer: (a) What is in M[8] when the CPU stops? Give it in decimal and in binary. (b) *Bonus:* what would happen if the PC ever reached address 6?

Task: solution

PC (cell read)	Bits	Decoded	AC after	M[8] after
0	0001 0110	LOAD 6	4	0
1	0011 0111	ADD 7	13	0
2	0011 0110	ADD 6	17	0
3	0010 1000	STORE 8	17	17
4	0111 0000	HALT	17	17

(a) $M[8] = 17 = 0001\ 0001$
(4 + 9 + 4: cell 6 holds 4, cell 7 holds 9)

(b) The CPU would fetch 0000 0100 and try to decode it as an instruction. Opcode 0000 is not in the table: a real CPU reports an *illegal instruction*. The bits mean “number 4” only because the program uses them as data.

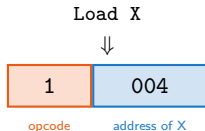
Typical mistakes: using the address as the value (ADD 6 adds the contents of cell 6, which is 4, not 6); reading the opcode from the wrong half; writing to M[8] before STORE; continuing after HALT.

Now a simulator is the CPU: MARIE

MARIE is a small CPU simulator designed for teaching. It has memory, registers, an accumulator, and the same fetch–decode–execute cycle we have just followed.

 <https://marie.js.org> (runs in the browser, nothing to install)

From a program instruction to machine code



1004

What the opcode does

The opcode tells MARIE **what operation to perform**. The remaining part identifies the operand or address. MARIE uses **16-bit** words: a 4-bit opcode and a 12-bit address, shown as 4 hexadecimal digits.

The simulator lets us observe the instruction, registers, memory, and output while the program runs.

Using MARIE.js

Four steps

1. Write

Type the program in the code editor.

2. Assemble

The assembler translates it into machine code and puts it in memory.

3. Step

Execute **one instruction** and look at the registers and memory.

4. Run

Execute the whole program until Halt.

What you will see

AC, PC, IR: the registers you already know.

MAR, MBR: address and data of the current memory access; **InREG, OutREG:** input and output.

Memory view: every cell in hexadecimal.

Output: values printed by the program.

Tip: choose **DEC** (decimal) for input and output.

Exercise 1: our program in MARIE



```
0  Load X      / AC <- M[X]
1  Add Y        / AC <- AC + M[Y]
2  Store Z      / M[Z] <- AC
3  Halt
4  X, DEC 5     / data
5  Y, DEC 3
6  Z, DEC 0
```

Line number = memory address.

X, DEC 5: cell named X holding the decimal value 5.

Text after / is a comment.

Explore the program in MARIE

Type the program and click **Assemble**.

- 1 Look at the Memory browser. In which memory cells are Load X, Add Y, Store Z and Halt?
- 2 What is stored in memory cells 4, 5 and 6?
- 3 Click **Step** once. Notice that **AC** changes from 0000 to 0005. Why?
- 4 Click **Step** again. What is the value of **AC** now, and why does it have this value?
- 5 Memory cell 6 still contains 0000. Click **Step** once more. What value does cell 6 contain now, and why did it change?

Solution: exercise 1

Memory after Assemble

Address	Contents	Meaning
0	1004	Load X
1	3005	Add Y
2	2006	Store Z
3	7000	Halt
4	0005	X
5	0003	Y
6	0000	Z

After the first Step: Load X reads the contents of cell 4 and puts 5 into the AC.

After the second Step: Add Y reads cell 5 and adds 3 to the AC: $5 + 3 = 8$.

After the third Step: Store Z writes 8 into cell 6. The program remains in cells 0–3.

New instructions: Subt, Input, Output

Instruction	Effect	Opcode	In words
Load X	$AC \leftarrow M[X]$	1	bring the contents of X into the AC
Store X	$M[X] \leftarrow AC$	2	write the AC into X
Add X	$AC \leftarrow AC + M[X]$	3	add the contents of X to the AC
Subt X	$AC \leftarrow AC - M[X]$	4	subtract the contents of X from the AC
Input	$AC \leftarrow$ value typed by the user	5	read a number from the user
Output	show the AC	6	print the value in the AC
Halt	stop	7	end of the program

grey rows: instructions you already know

```
0  Input      / read a number
1  Subt One   / AC <- AC - 1
2  Output     / print it
3  Halt
4  One, DEC 1
```

Example: prints the number typed by the user, minus 1. MARIE has no “subtract 1” instruction, so we keep the value 1 in a cell named One.

Exercise 2: sum two inputs

Task

Read two numbers from the user and print their sum.

Test: 7 and 5 $\rightarrow$ 12

Hint: the AC holds only one value. Store the first input before reading the second one. Every variable needs a memory cell, for example A, DEC 0.

Solution: exercise 2



```
0  Input
1  Store A
2  Input
3  Add A
4  Output
5  Halt
6  A, DEC 0
```

For inputs 7 and 5:

first input: $AC = 7$, then Store A;

second input: $AC = 5$;

Add A: $AC = 5 + 7 = 12$.

Exercise 3: calculate $2x - y$

Task

Read x and then y from the user and print $2x - y$.

Test: $x = 10, y = 3 \rightarrow 17$

Hint: save both inputs. There is no multiplication instruction, so calculate $2x$ as $x + x$.

Solution: exercise 3



```
0  Input
1  Store X
2  Input
3  Store Y
4  Load X
5  Add X
6  Subt Y
7  Output
8  Halt
9  X, DEC 0
10 Y, DEC 0
```

For $x = 10$, $y = 3$:

Load X; Add X: $AC = 20$;

Subt Y: $AC = 20 - 3 = 17$.

New instructions: Jump and Skipcond

So far, the PC has moved to the next memory cell after each instruction. Now we introduce two instructions that can **change this normal flow**. Jump changes where the PC goes; Skipcond can skip the next instruction.

Instruction	Effect
Jump X	$PC \leftarrow X$
Skipcond 000	skip the next instruction if $AC < 0$
Skipcond 400	skip the next instruction if $AC = 0$
Skipcond 800	skip the next instruction if $AC > 0$

Loop, is a label. Jump Loop sends the PC back to that instruction. Skipcond checks the AC; when the condition is true, it skips the next instruction. A complete example follows on the next slide.

How the loop works: Jump and Skipcond



```

0      Input
1      Store N
2  Loop, Load N
3      Output
4      Subt One
5      Store N
6      Skipcond 400 / N = 0?
7      Jump Loop   / no: repeat
8      Halt        / yes: stop
9  N,    DEC 0
10 One,  DEC 1
    
```

What happens at the end of each pass?

- ① Subt One decreases N by 1.
- ② Skipcond 400 checks whether the AC is zero.
- ③ If AC is not zero, Jump Loop repeats the loop.
- ④ If AC is zero, Skipcond skips Jump Loop and the PC reaches Halt.

For input 3, the output is 3, 2, 1. The loop stops when N becomes 0. Works for input ≥ 1 : for 0, N becomes negative and never reaches 0.

Exercise 4: a loop

Exercise 4: sum from 1 to n

Read a number $n \geq 1$ and print $1 + 2 + \dots + n$.

Test: $n = 4 \rightarrow 10$

Hint: add N to a running sum, decrease N , and use `Skipcond 400` to stop when it reaches 0.

If your program never stops, press **Stop**: check the condition in `Skipcond` and what is in the AC at that moment.

Solution: exercise 4 (sum from 1 to n)

```

0      Input
1      Store N
2  Loop, Load N
3      Add Sum
4      Store Sum
5      Load N
6      Subt One
7      Store N
8      Skipcond 400 / N = 0?
9      Jump Loop    / no: repeat
10     Load Sum     / yes: print
11     Output
12     Halt
13  N,   DEC 0
14  One, DEC 1
15  Sum, DEC 0
  
```

Run for $n = 4$

Pass	Sum after	N after	Skipcond 400
1	4	3	AC $\neq$ 0: jump back
2	7	2	AC $\neq$ 0: jump back
3	9	1	AC $\neq$ 0: jump back
4	10	0	AC = 0: skip Jump

After the fourth pass Skipcond skips Jump Loop, so the program prints 10 and halts.